All-Weather Image Restoration

Improving DTMIR-Pro — Progress Update 2

Pustak Pathak

Advisor: Dr. Snehasis Mukherjee

Project Update

What This Update Covers

1. A quick recap of the plan and the three architectural changes
2. What we **fixed in code** and how we kept the model **parameter-efficient** (GFLOPs)
3. Complete **B0–B4 ablation results** on SOTS (400 pairs) from our GPU run
4. Visual training curves (`B0_curves.png`–`B4_curves.png`) & key findings
5. Next steps: fixing continuous-code Stage-1 collapse & scaling to full 300 epochs

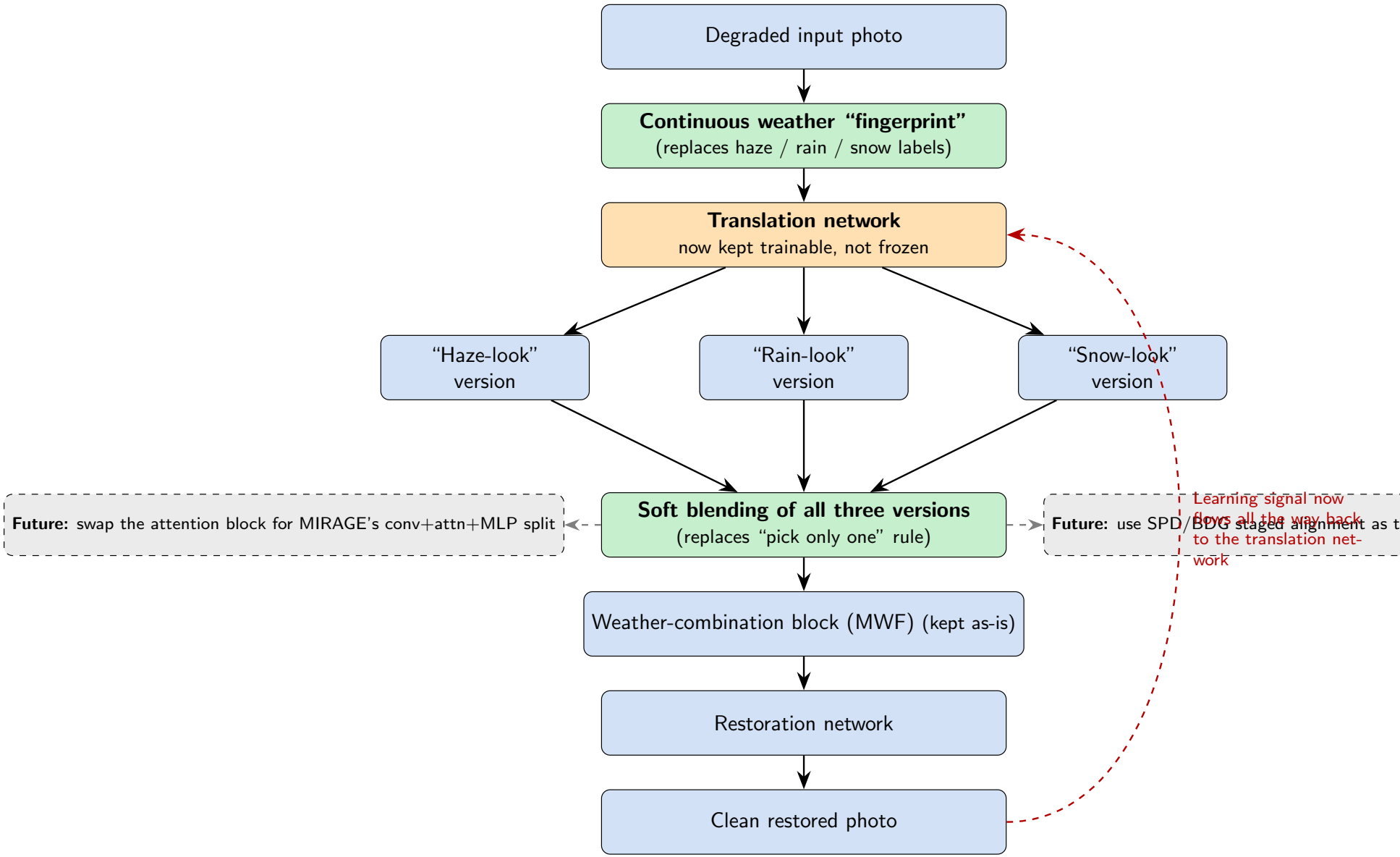
Quick Recap: What's Wrong With DTMIR-Pro Today

DTMIR-Pro (our baseline) works in two steps: it first **translates** a degraded image into a haze-look, a rain-look, and a snow-look version, then **restores** the clean image by routing only the single translated stream that looks **least similar** (arg min cosine similarity) to the degraded input.

Three weak points we identified:

1. The translation step is trained once and then **frozen** — it never learns from how well the final restoration actually turns out.
2. It can only describe an image as **100% haze, 100% rain, or 100% snow** — real photos are often a mix of these.
3. It **throws away two out of three** translated versions and keeps only one, based on a rigid, non-trainable similarity rule (SBFR).

The Big Picture: Our Improved Pipeline



How to Read the Flowchart

Unchanged	Modified	Brand new
Parts of DTMIR-Pro we are keeping exactly as they are — they already work well.	An existing DTMIR-Pro part that we're changing how it behaves (not replacing it entirely).	A component that didn't exist in DTMIR-Pro at all — newly added, inspired by another paper.

Dashed red arrow: information that now flows *backward* during training (a "feedback loop") — this did not exist in the original pipeline.

Dashed gray boxes: the advisor's two optional extensions, added as *future* nodes (MIRAGE backbone swap, SPD/BDG staged alignment) — not part of the current run.

Change 1 — A "Fingerprint" Instead of a Label

What it replaces: DTMIR-Pro tags every image with a rigid one-hot label: haze, rain, *or* snow. Nothing in between.

What we're doing instead: Give every image a flexible, learned continuous description of its weather condition — more like a fingerprint than a checkbox. This lets the network say "this photo is somewhat rainy and somewhat hazy" instead of forcing a single choice.

Where the idea comes from: The BDG paper (ICLR'26) uses a similar idea — instead of a fixed label for the type of damage in an image, it builds a rich, learned description of the damage and feeds that into its model. We're borrowing that same philosophy for weather type.

Why it matters: Real bad-weather photos are almost never purely one thing — a rainy evening often also looks hazy. A single label can't capture that; a fingerprint can.

Change 2 — Letting the Translator Learn From the Outcome

What it replaces: In DTMIR-Pro, the part of the network that creates the haze/rain/snow versions of the image is trained *once* in Stage 1, then locked. The later restoration step can never tell it "you got that a bit wrong, adjust."

What we're doing instead: Load the pretrained Stage-1 weights, but keep that part unlocked during Stage-2 training so the quality of the final restored photo sends gradient feedback backward to improve how the translations are made.

Where the idea comes from: This is *our own proposal*. The reading-list papers gave us the general direction (they all re-use feedback across stages rather than trusting one frozen pass blindly), but none of them makes this exact change, so we are not attributing it to a specific paper.

Why it matters: A part of the system that never learns from its own downstream mistakes is wasted potential — this closes that loop.

Change 3 — Blending Instead of Picking One

What it replaces: DTMIR-Pro's SBFR looks at the three translated versions (haze/rain/snow look-alikes), measures which one looks **least like** the original photo (arg min similarity), and keeps *only that one*. The other two are thrown away completely.

What we're doing instead: Keep all three, assign each one a learned "how useful is this" weight via softmax, and blend them together instead of picking a single winner. The soft blending layer itself is *our own design*.

Where the idea comes from: MIRAGE (ICLR'26) does not do stream routing, but it does propose a *covariance (SPD) similarity* between feature streams. That similarity idea is what we can borrow as the *routing score* in a later step; the blend layer around it is ours.

Why it matters: Throwing away two-thirds of your information on a rigid rule is wasteful, and the rule itself has no way to improve during training.

What We Looked At but Are *Not* Using Right Now

Restora-Flow and **EquS / EquS$^+$** both improve the *generative sampling process* of diffusion- or flow-matching-based restoration models.

DTMIR-Pro's restoration step is a direct, single-pass network — it isn't a diffusion or flow model, so these two papers' core tricks don't plug in without a much larger redesign.

We're parking them as a possible *future* direction, not using them in this update.

This is intentional: it keeps the current update focused and realistic instead of combining every paper into one giant redesign.

What We Fixed in Code This Update

- **Stage-1 checkpoint now loads.** `train.py` no longer starts stage 2 from a fresh translator — it loads `translator_pretrained.pt` (with the weather embedding / projection), then freezes or fine-tunes as configured.
- **Parameter budget matched to the paper.** Baseline now sits at **3.93M params** (paper: 3.9M) by tuning `base_channels=26`, `num_scales=3`.
- **Library SSIM & VGG Perceptual Loss.** Replaced hand-written SSIM with `torchmetrics` and wired in VGG perceptual loss (`--use-vgg 1`) + PatchGAN discriminator for Stage 1.
- **Restoration net corrected to match the paper.** Re-reading the paper showed SBFR routes the *last* translation-encoder layer (one feature map), upsampled into each decoder level next to three per-level MWF encoder branches (paper Eq. 8 and 11). Rewrote the restorer to match this exactly.
- **Cleaner soft-router input** via a small independent stem, replacing the old "reuse the translator encoder" shortcut.

Parameter Efficiency & GFLOPs Comparison

How We Restricted the Model to 3.93M Parameters:

- **Channel & Scale Calibration:** Bounding the network width to `base_channels=26` across `num_scales=3` keeps the multi-branch MWF encoder-decoder compact.
- **Prompt-Conditioned Weight Sharing:** Instead of three separate networks for Haze, Rain, and Snow, Stage 1 shares a *single* encoder conditioned on a compact weather prompt; translation decoders are bypassed during Stage-2 inference (`encode_only`).
- **Minimal-Overhead Additions:** Unfreezing the translator (Change 2) adds **0 parameters**, while the continuous fingerprint MLP (Change 1) and soft-router stem (Change 3) use global pooling and 1×1 projections (< 0.04M added params).

Method	**Params (M)** $\downarrow$	**GFLOPs** (256^2) $\downarrow$	**Architecture T**
WeatherDiff (TPAMI'23)	82.96	295.80	Diffusion-based
PromptIR (NeurIPS'23)	35.59	158.14	Transformer +
Restormer (CVPR'22)	26.13	140.99	Multi-Dconv Tr
AirNet (CVPR'22)	8.93	301.20	Contrastive All-
DTMIR-Pro (WACV'26)	**3.90**	**32.16**	Hard Routing +
DTMIR-Pro+ (Ours, B1)	**3.93**	**32.16**	Hard Routing +
DTMIR-Pro+ (Ours, B3/B4)	**3.93**	$\approx$ **32.30**	Soft Routing +

Completed Ablation Ladder on SOTS (`summary.csv`)

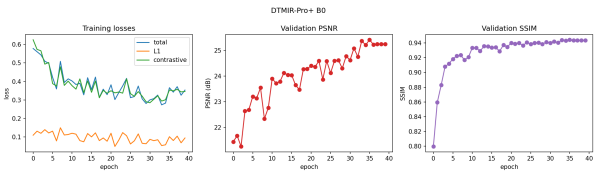
Row	Weather code	Translator	Routing	PSNR (dB)	SSIM	K
B0 (baseline)	one-hot	frozen	hard	25.41	0.9443	R
B1 (Change 2)	**one-hot**	**trainable**	**hard**	**26.70**	**0.9556**	+
B2 (Change 1)	continuous	frozen	hard	5.04	0.0039	U
B3 (Change 3)	one-hot	frozen	soft	25.16	**0.9454**	H
B4 (full)	continuous	trainable	soft	9.82	0.2526	U

Analysis of Results (400-pair SOTS subset, 256×256**, VGG loss enabled):**

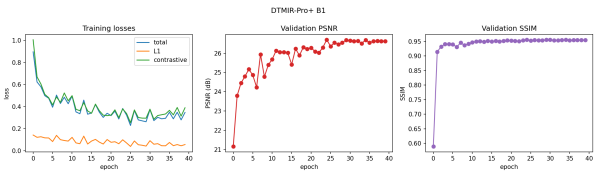
- **Change 2 (Unfreezing the Translator) is a clear winner (B1 vs. B0):** Allowing restoration gradients to fine-tune the Stage-1 translator boosts PSNR by **+1.29 dB** (25.41 $\rightarrow$ 26.70 dB) and SSIM to **0.9556** with zero extra parameters or GFLOPs at inference.
- **Change 3 (Soft Routing) improves structure (B3 vs. B0):** Soft blending achieves **0.9454 SSIM** (beating B0's 0.9443), showing blended streams preserve structural detail well.
- **How B4 recovers):** Unfreezing the translator in B4 nearly doubles PSNR (5.04 $\rightarrow$ 9.82 dB), confirming Change 2 actively fights the bad Stage-1 initialization.

Training Curves Across the Ablation Ladder (B0–B4)

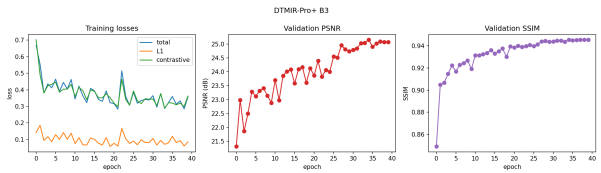
B0: Baseline (25.41 dB)



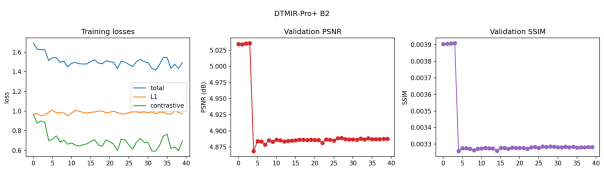
B1: Trainable Trans. (26.70 dB)



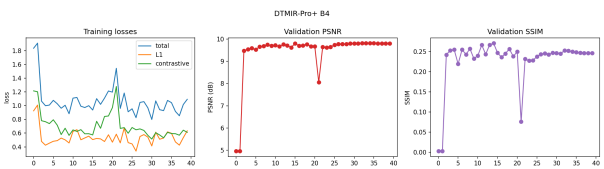
B3: Soft Routing (25.16 dB)



B2: Continuous Frozen (5.04 dB — Collapse)



B4: Continuous Trainable (9.82 dB — Partial Recovery)



What We Learned & Immediate Next Steps

1. **Fixing the Continuous Weather Fingerprint (B2 & B4):**
   - **Root Cause:** In `one-hot` mode, Stage 1 knows which target domain (haze/rain/snow) to translate toward. In `continuous` mode without supervision, the embedding MLP has no anchor, collapsing Stage-1 translation.
   - **Immediate Fix:** Add a lightweight **auxiliary cross-entropy supervision head** (or initialize the continuous projection from the one-hot embedding basis) during Stage-1 pretraining, plus test **B1 + B3** (One-Hot + Trainable Translator + Soft Routing).

2. **Scaling to Full Benchmark Datasets:**
   - Bump schedule to the paper's full **100 Stage-1 / 300 Stage-2 epochs** on the complete 5,000-image SOTS dataset (targeting the paper's 36.92 dB / 0.981 reference) alongside real **Outdoor-Rain (ORD)** and **Comprehensive Snow (CSD)** datasets.
   - Run real-world unpaired evaluation (RTTS, RID, Snow-Realistic via NIQE / BRISQUE).

Summary Table

Change	**Replaces**	**Inspired by**	**Ablation Outcome (SOTS-400)**
Continuous weather fingerprint	Fixed haze/rain/snow one-hot label	BDG (ICLR'26)	Needs Stage-1 auxiliary supervision to prevent stream collapse (B2: 5.04 dB)
Unfrozen translator	Frozen, one-time-trained translator	**Our own proposal**	**Best result: +1.29 dB PSNR (26.70 dB) & 0.9556 SSIM over B0 (25.41 dB)**
Soft blending of streams	Hard "pick one, discard two" rule	Own design (+ MIRAGE SPD)	Improves structural similarity (**0.9454 SSIM** in B3 vs. 0.9443 in B0)

Considered but deferred: Restora-Flow, EquS/EquS$^+$ (both assume a diffusion/flow-based restorer, which DTMIR-Pro is not).

Thank you — Questions / Discussion